🚀 Day 21 – Hashing Problems (DSA) Part-1

🔥 Hashing Recap (Quick)

Hashing ka use hota hai:

Fast lookup (O(1))

Frequency count

Duplicate detection

Pair finding

👉 Mostly use hota hai:

HashMap

HashSet

🧠 Problem 1: Frequency of Elements

📌 Problem

Array me har element kitni baar aaya hai, find karo.

💡 Approach

Ek HashMap lo

Key = element

Value = count

✅ Code (Java)

```
import java.util.*;

public class Frequency {
    public static void main(String[] args) {
        int[] arr = {1, 2, 2, 3, 1, 4, 2};

        HashMap<Integer, Integer> map = new HashMap<>();

        for(int num : arr){
            map.put(num, map.getOrDefault(num, 0) + 1);
        }

        System.out.println(map);
    }
}
```

⏱️ Time Complexity

O(n)

🧠 Problem 2: Two Sum (Important 🔥)

📌 Problem

Target sum ke liye 2 numbers ke indices find karo.

💡 Approach

Complement find karo

Map me check karo

✅ Code

```
import java.util.*;

public class TwoSum {
    public static int[] twoSum(int[] arr, int target){
        HashMap<Integer, Integer> map = new HashMap<>();

        for(int i = 0; i < arr.length; i++){
            int complement = target - arr[i];

            if(map.containsKey(complement)){
                return new int[]{map.get(complement), i};
            }

            map.put(arr[i], i);
        }

        return new int[]{-1, -1};
    }
}
```

⏱️ Complexity

Time: O(n)

Space: O(n)

🧠 Problem 3: First Unique Character

📌 Problem

String me first non-repeating character find karo.

💡 Approach

Frequency count karo

Phir first unique find karo

✅ Code

```
import java.util.*;

public class FirstUnique {
    public static char firstUnique(String s){
        HashMap<Character, Integer> map = new HashMap<>();

        for(char ch : s.toCharArray()){
            map.put(ch, map.getOrDefault(ch, 0) + 1);
        }

        for(char ch : s.toCharArray()){
            if(map.get(ch) == 1){
                return ch;
            }
        }

        return '_';
    }
}
```

🧠 Problem 4: Anagram Check

📌 Problem

Do strings anagram hai ya nahi?

👉 Example:

"listen" = "silent" ✅

💡 Approach

Character frequency compare karo

✅ Code

```
1. import java.util.*;

public class Anagram {
    public static boolean isAnagram(String s1, String s2){
        if(s1.length() != s2.length()) return false;

        HashMap<Character, Integer> map = new HashMap<>();

        for(char ch : s1.toCharArray()){
            map.put(ch, map.getOrDefault(ch, 0) + 1);
        }

        for(char ch : s2.toCharArray()){
            if(!map.containsKey(ch)) return false;
            map.put(ch, map.get(ch) - 1);
            if(map.get(ch) == 0){
                map.remove(ch);
            }
        }

        return map.isEmpty();
    }
}
2.
```

🧠 Problem 5: Subarray Sum = K (VERY IMPORTANT 🔥 🔥)

📌 Problem

Count number of subarrays jinka sum = k ho.

💡 Approach (Prefix Sum + HashMap)

👉 Ye interview ka favourite hai

Prefix sum calculate karo

Check karo: (currentSum - k) map me hai ya nahi

✅ Code

```
import java.util.*;

public class SubarraySum {
    public static int subarraySum(int[] arr, int k){
        HashMap<Integer, Integer> map = new HashMap<>();
        map.put(0, 1);

        int sum = 0;
        int count = 0;

        for(int num : arr){
            sum += num;

            if(map.containsKey(sum - k)){
                count += map.get(sum - k);
            }

            map.put(sum, map.getOrDefault(sum, 0) + 1);
        }

        return count;
    }
}
```

🔥 Example

arr = [1,1,1], k = 2
Output = 2

⏱️ Complexity

Time: O(n)

Space: O(n)

🧠 Bonus Problem: Longest Consecutive Sequence

📌 Problem

Longest consecutive sequence length find karo.

💡 Approach

HashSet use karo

Sequence start detect karo

✅ Code

```java
import java.util.*;

public class LongestConsecutive {
    public static int longestConsecutive(int[] nums){
        HashSet<Integer> set = new HashSet<>();

        for(int num : nums){
            set.add(num);
        }

        int longest = 0;

        for(int num : set){
            if(!set.contains(num - 1)){
                int current = num;
                int count = 1;

                while(set.contains(current + 1)){
                    current++;
                    count++;
                }

                longest = Math.max(longest, count);
            }
        }

        return longest;
    }
}
```

🎯 Summary (Interview Ready)

Problem	Concept
Frequency	Counting
Two Sum	Complement
First Unique	Frequency + traversal
Anagram	Frequency match
Subarray Sum	Prefix Sum
Longest Sequence	HashSet